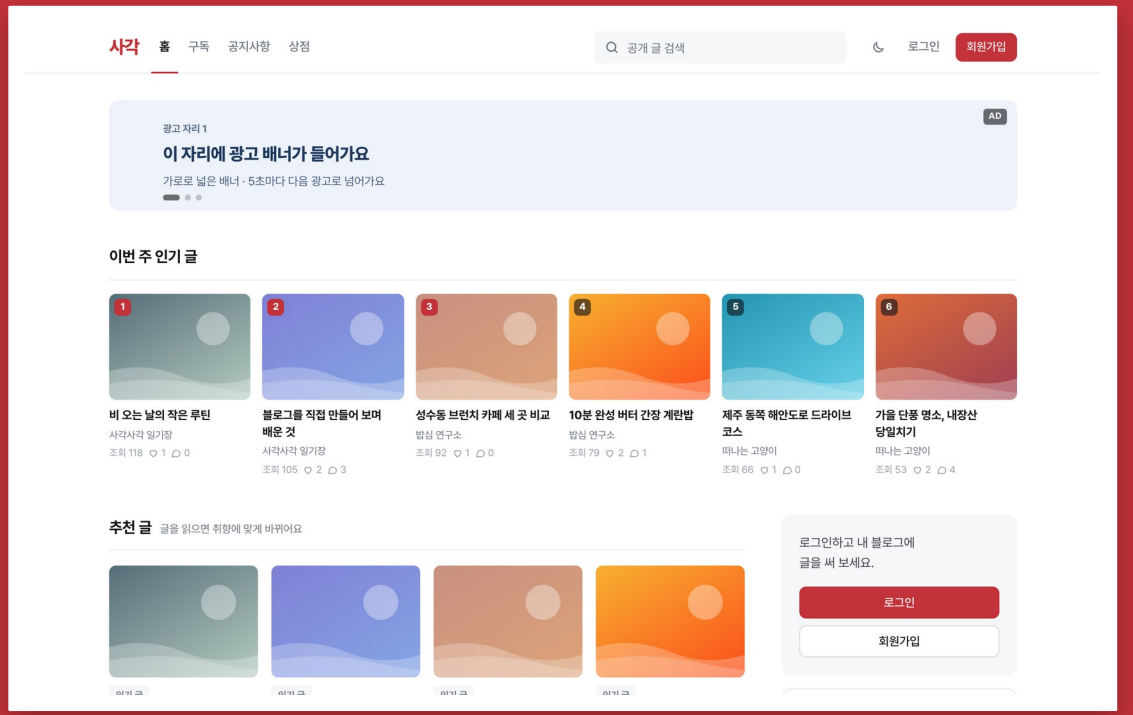
NHN 아카데미 · AI 프로젝트 · 개인 프로젝트 소개자료

사각

회원마다 블로그 하나, 그 안에 다이어리 여러 권
꾸미며 쓰는 다이어리형 블로그 서비스



박수연 AIGJ_01_010

사이트 https://sagak.java21.net

GitHub github.com/home-my-blog/myblog

제출 저장소 github.com/AIP-1/blog-basic-AIGJ_01_10_pblog

목차

사각 개인 프로젝트 소개자료

01 **프로젝트 개요**

주제와 기획의도 · 개발환경 · 구조(구성도 · 패키지 · ERD) · 기대 효과

02 **프로젝트 구현 범위**

기능별 구현 내용과 완성 여부(완료 · 일부 완료)

03 **프로젝트 수행 절차 및 방법**

기획 → 설계 → 개발 → 테스트 · 배포, 9일간의 일정

04 **프로젝트 수행 경과**

주요 기능 화면 · 기술적 고민 6가지 · 트러블슈팅 6건 · AI 활용

05 **자체 평가 의견**

완성도 · 잘한 점과 아쉬운 점 · 개선할 점 · 느낀 점

① 주제 · 선정 배경 · 기획의도

“블로그를 다이어리 꾸미기처럼” — 회원마다 블로그가 하나 생기고, 블로그 안에 다이어리(분류)를 여러 권 만들어 글을 쓴다. 활동하면 잉크가 쌓이고, 잉크로 스티커를 사서 다이어리 표지를 꾸민다.

선정 배경

- 개인 프로젝트로, 블로그 서비스에 꼭 필요한 기본 기능을 공통 요구사항 (CF-01~24)으로 먼저 정리했다.
- 기본 기능만으로는 흔한 블로그가 된다. 글 목록 중심의 일반 블로그 대신, 꾸준히 쓰고 싶게 만드는 장치에 집중했다.
- 혼자 기획부터 배포 · 운영까지 해 보며, 수업에서 배운 세션 인증 · 리버스 프록시 · CI/CD · 테스트 전략을 실제 서비스에 적용하는 것이 목표였다.

기존 블로그와 다른 점

1. **다이어리 단위 운영**

   다이어리마다 공개 · 구독자만 · 비공개를 정하고, 블로그 전체 또는 다이어리 한 권만 구독할 수 있다.

2. **쓰는 동기를 주는 꾸미기**

   출석 · 글 · 댓글로 잉크를 모아 스티커 팩을 사고, 다이어리 표지에 붙인다. 도전과제도 있다.

3. **읽는 재미와 안전한 소통**

   5초마다 바뀌는 실시간 인기 검색어, 쓴 사람과 블로그 주인만 보는 비밀 댓글.

② 구현 내용과 컨셉

컨셉: 내 다이어리를 꾸미며 쓰는 블로그 — 사이트는 붉은 포인트 색 하나로 통일하고, 야간모드도 지원한다

1 글쓰기

마크다운 에디터 · 미리보기
사진 10장 · 태그 5개
60초 자동 임시저장 · 대표글 고정

2 읽기와 소통

댓글 · 답글 · 비밀 댓글
좋아요 · 책갈피 폴더
블로그 / 다이어리 구독 피드

3 탐색

분야별 글(최신 · 인기순)
통합 검색(한국어 부분 일치)
실시간 인기 검색어 · 인기 글

4 꾸미기

잉크 적립(출석 · 글 · 댓글)
스티커 상점 · 표지 꾸미기
도전과제 · 야간모드

5 회원

이메일 인증 가입
카카오 · 구글 로그인
비밀번호 찾기 · 30일 안에 복구

6 블로그 관리

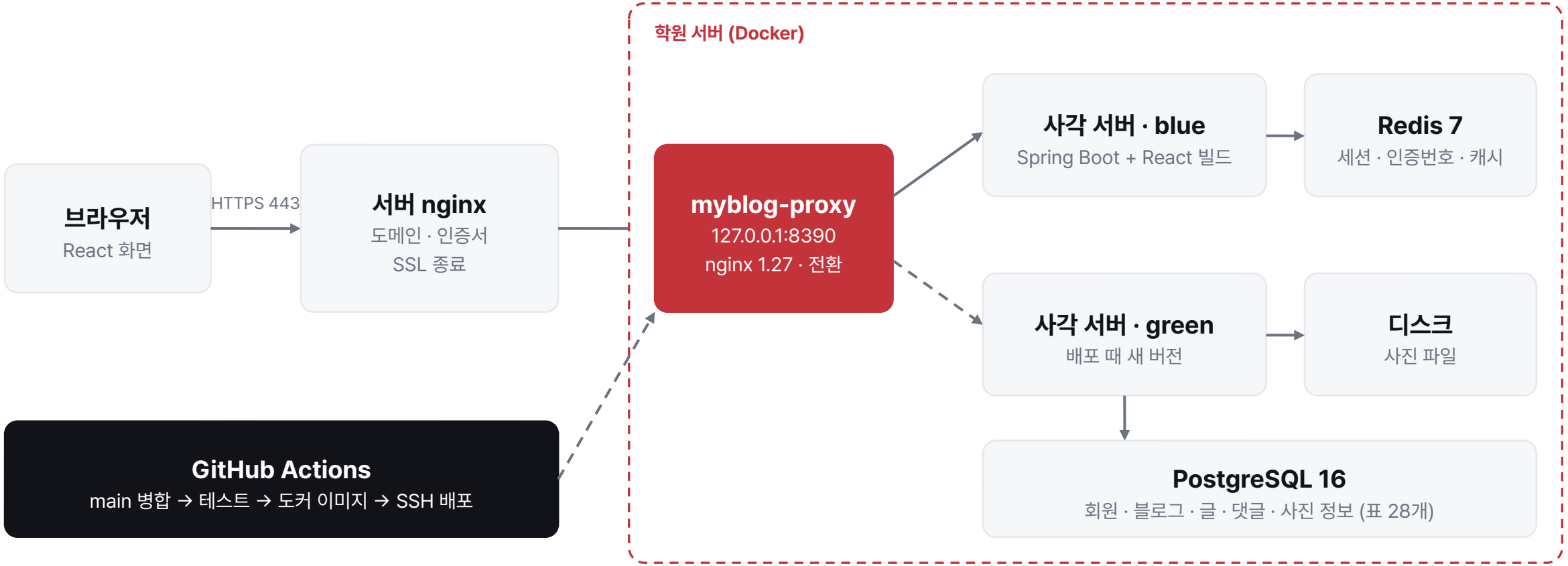
대시보드 · 7일 / 30일 통계
글 · 댓글 모아 보기
다이어리 순서 · 공개 범위

훈련 내용과의 연관성: Spring Boot · React 개발, 세션과 쿠키 인증, Docker · nginx 리버스 프록시, GitHub Actions CI/CD, 테스트 피라미드와 정적 분석을 모두 사각에 적용했다.

③ 활용 장비 및 재료 — 개발환경과 버전

영역	사용 기술 (버전)	고른 이유
언어	Java 21 · TypeScript 5.9	지원 중인 LTS, 타입으로 API 응답 모양 실수를 컴파일 단계에서 차단
서버	Spring Boot 3.5.14 (Security · Session · OAuth2 Client) · JdbcClient · Flyway	인증 · 세션 도구가 갖춰져 있음. SQL을 코드에 그대로 보이게 JPA 대신 JdbcClient
화면	React 19.2 · React Router 7 · TanStack Query 5 · Vite 7 · react-markdown 10	컴포넌트 단위 화면, 서버 데이터 캐시를 라이브러리에 맡김
데이터	PostgreSQL 16 (pg_trgm) · Redis 7	강하게 연결된 데이터는 관계형 DB, 곧 사라질 값 · 세션 · 캐시는 Redis
빌드 · 테스트	Gradle 8.14 · Node 22 · JUnit 5 · Vitest 5 · Testing Library · Playwright 1.56 · JaCoCo	단위 · 통합 · 종단(E2E) 테스트와 커버리지
품질	SonarQube Cloud · ESLint 9 · tsc	품질 게이트를 통과해야만 병합
배포	Docker (eclipse-temurin 21) · nginx 1.27 · GitHub Actions · 서버 nginx(HTTPS)	같은 이미지로 로컬과 서버를 맞추고, 블루-그린 무중단 배포
개발 도구	Claude Code · Spec Kit · Git / GitHub · Notion · Crowfoot(ERD)	AI 페어 프로그래밍, 명세 → 계획 → 작업 순서로 개발

④ 프로젝트 구조 — 시스템 구성도



앱 포트(8390)는 서버 안에서만 열고, 바깥은 HTTPS 도메인으로만 들어온다. 화면과 API를 같은 주소로 내보내 쿠키 세션을 그대로 쓰고 CORS 설정이 필요 없다.

④ 프로젝트 구조 — 패키지 구조와 ERD

backend/ (com.myblog)
├ member 가입 · 로그인 · 소셜 · 탈퇴
├ blog 블로그 · 다이어리 · 구독
├ post 글 · 임시저장 · 태그 · 좋아요
├ comment 댓글 · 비밀 댓글
├ search 검색 · 인기 검색어
├ shop 잉크 · 스티커 · 도전과제
├ image 사진 (디스크 / S3)
├ manage 블로그 관리 · 통계
├ home · stats · mail · config
└ common 오류 응답 · RedisGuard
db/migration Flyway V1 ~ V25

frontend/src
├ pages · components · api · auth
└ messages.ts 화면 문구 모음



핵심 표만 그린 그림. 전체는 표 28개 · 외래 키 45개 (Crowfoot ERD, docs/설계/ERD.md)

⑤ 활용 방안과 기대 효과

1 바로 쓰는 개인 기록 서비스

실제 도메인(sagak.java21.net)에서 HTTPS로 운영 중이다. 가입부터 글쓰기 · 구독 · 꾸미기까지 혼자서도, 여럿이서도 쓸 수 있다.

2 따라 할 수 있는 운영 사례

무중단 배포 스크립트, CI 품질 게이트, 테스트 전략, 기능 · 원리 문서를 공개 저장소에 남겨 다른 프로젝트의 출발점으로 쓸 수 있다.

3 확장 가능한 구조

세션이 이미 Redis에 있어 앱을 두 대로 늘려도 코드 변경이 없다. 사진 저장소는 인터페이스로 분리해 설정만으로 S3 호환 저장소로 바꾼다.

154배

메인 화면 처리량

초당 18.6건 → 2,868건 (Redis 응답 캐시)

0회

배포 중 끊김

0.2초 간격 요청 중 배포, 로그인 유지

89.7%

테스트 커버리지

서버 127 · 화면 297 · E2E 7개

성능 수치는 로컬에서 글 20,769개를 넣고 Apache Bench(동시 10명)로 같은 조건에서 캐시 전후를 잰 값이다.

기능별 구현 내용과 완성 여부

기능	구현 내용	상태
회원	이메일 인증 가입 · 로그인(5회 실패 10분 잠금) · 비밀번호 찾기 · 카카오 / 구글 로그인 · 탈퇴 후 30일 안에 복구	완료
블로그 · 다이어리	블로그 설정 · 분야, 다이어리 추가 · 순서 · 공개 범위(공개 / 구독자만 / 비공개)	완료
글쓰기	마크다운 · 미리보기, 사진 10장, 태그 5개, 60초 자동 임시저장, 대표글 3개 고정	완료
읽기 · 소통	댓글 · 한 단계 답글 · 비밀 댓글, 좋아요, 블로그 / 다이어리 구독과 피드, 책갈피 폴더	완료
탐색	분야별 글(최신 · 인기순), 통합 검색, 실시간 인기 검색어, 이번 주 인기 글 · 인기 다이어리	완료
꾸미기	잉크 적립 · 스티커 상점 · 다이어리 표지 꾸미기 · 도전과제 · 야간모드	완료
블로그 관리	대시보드 · 7일 / 30일 통계 그래프 · 글 / 댓글 모아 보기 · 새 댓글 표시	완료
운영	HTTPS 도메인, 자동 배포, 블루-그린 무중단 배포, 공개 화면 캐시, 품질 게이트	완료
신고	사유 4가지로 신고 저장, 한 글에 한 번 (신고를 처리하는 화면은 없음)	일부 완료
알림	토스트 · 잉크 적립 알림 · 새 댓글 수 배지 (서버가 먼저 보내는 푸시 알림은 없음)	일부 완료

9일 동안 릴리스 21번 — 기획 → 설계 → 개발 → 테스트 · 배포



Spec Kit으로 명세를 먼저 쓰고(왼쪽 네 칸), 기능마다 같은 순서로 구현 · 검증 · 배포를 반복했다(오른쪽 네 칸).

기간	단계	활동	버전
10.01~02	사전 기획	화면 기준 요구사항과 공통 기능(CF-01~24)을 정리하고, 사각의 주제 · 컨셉(다이어리 꾸미기)을 정함	-
10.08	설계 · 첫 배포	공통 요구사항 → Spec Kit(spec · plan · tasks), 프로젝트 원칙 6가지, 서버 · 화면 구현, 첫 배포	v1.0.0
10.12	기능 확장	도메인 연결, 사각 디자인, 분야 · 블로그 소개, 구독 · 책갈피, 야간모드, 스티커 상점, 인기 검색어	v1.1 ~ v1.6.1
10.13	운영 기반	블루-그린 무중단 배포 + Redis 세션, 보안 수정, 카카오 · 구글 로그인	v1.7 ~ v1.8.1
10.14	성능 · 품질 · 보안	가입 방법별 로그인 정책, 공개 화면 캐시(154배), SonarQube 품질 게이트, 앱 포트 숨기기 + HSTS	v1.9 ~ v1.10.1
10.15	품질 정리 · 테스트	SonarQube 이슈 229개 정리, 화면 테스트 241개 추가(커버리지 34% → 94%)	v1.10.2
10.16	마무리	E2E 테스트 · TESTING.md, 비밀 댓글, README · 문서 · ERD 정리	v1.10.3 ~ v1.11.1
총 개발기간	**10.08 ~ 10.16 (9일)**	병합한 PR 83개 · 브랜치 feature/* → develop → main · CHANGELOG가 바뀌면 GitHub Release 자동 생성	**릴리스 21개**

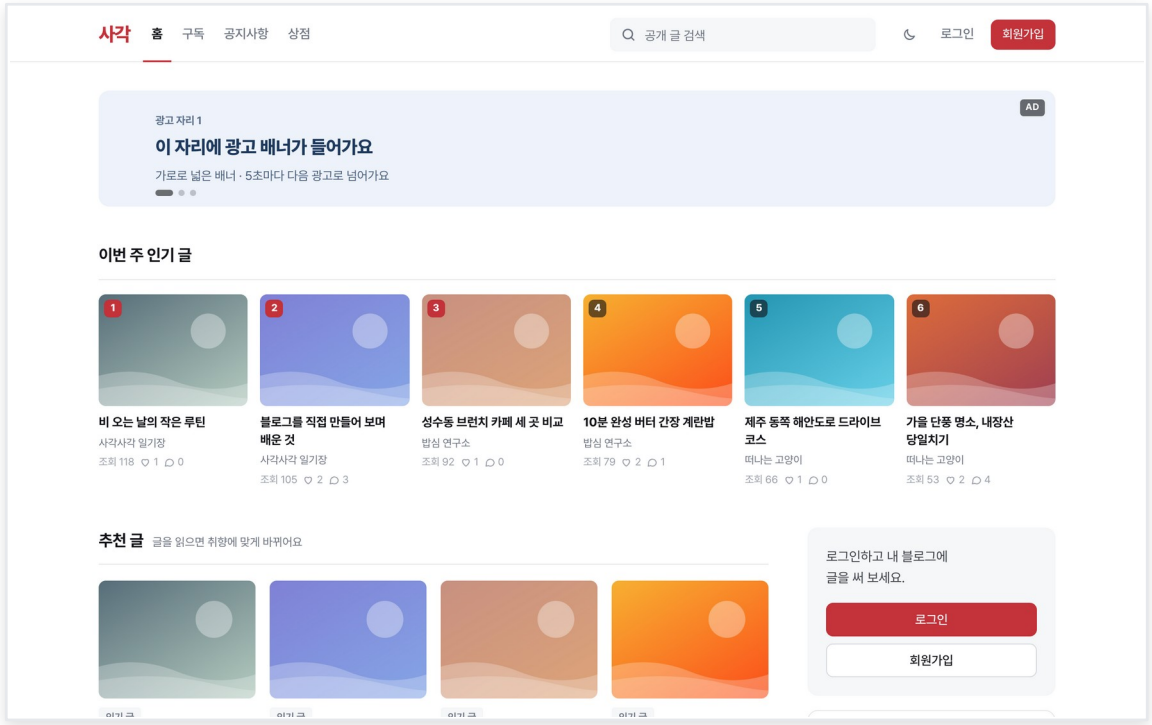
결과물이 나오기까지 — 피드백을 받아 고친 과정

만든 뒤 끝내지 않고, 사용자 시나리오 점검 · 수업 · 자동 검사에서 나온 피드백을 다음 버전에 바로 반영했다.

피드백을 얻은 곳	발견한 것	반영한 것	버전
사용자 시나리오 점검	소셜 계정으로 탈퇴한 뒤 다시 로그인하면 의도와 달리 바로 복구됨	사용자가 선택할 수 있게 복구 전에 한 번 묻도록 변경	v1.9.2
전문가 피드백 (리버스 프록시)	앱 포트 8390이 모든 주소에 열려 HTTPS를 건너뛰고 접속 가능	127.0.0.1에만 바인딩 + HSTS, 외부망에서 확인	v1.10.1
SonarQube 품질 게이트	이메일 정규식의 ReDoS 위험, 새 코드 커버리지 61%	소유 수량자로 수정 + 테스트 추가 후 병합	v1.10.2
수업 (테스트 전략)	테스트 피라미드, AI가 쓴 테스트를 믿지 말 것	E2E 7개, TESTING.md · CLAUDE.md 규칙	v1.10.3
배포 후 실제 흐름 점검	세션을 Redis로 옮긴 뒤 두 번째 요청부터 로그인이 풀림 (자동 테스트는 통과)	쿠키 형식 수정 + 재요청 테스트 추가로 로그인 유지	v1.7.0
사용자 편의성 (UI 일관성)	비밀 댓글 표시만 이모지라 다른 아이콘과 톤이 달라 눈에 덜 띔	사이트 아이콘과 같은 선 모양 자물쇠로 통일해 한눈에 알아보게	v1.11.1

이어지는 슬라이드: ⑤ 주요 기능 화면 → ⑥ 기술적 고민 → ⑦ AI 활용

메인 화면과 글쓰기



① 메인 화면

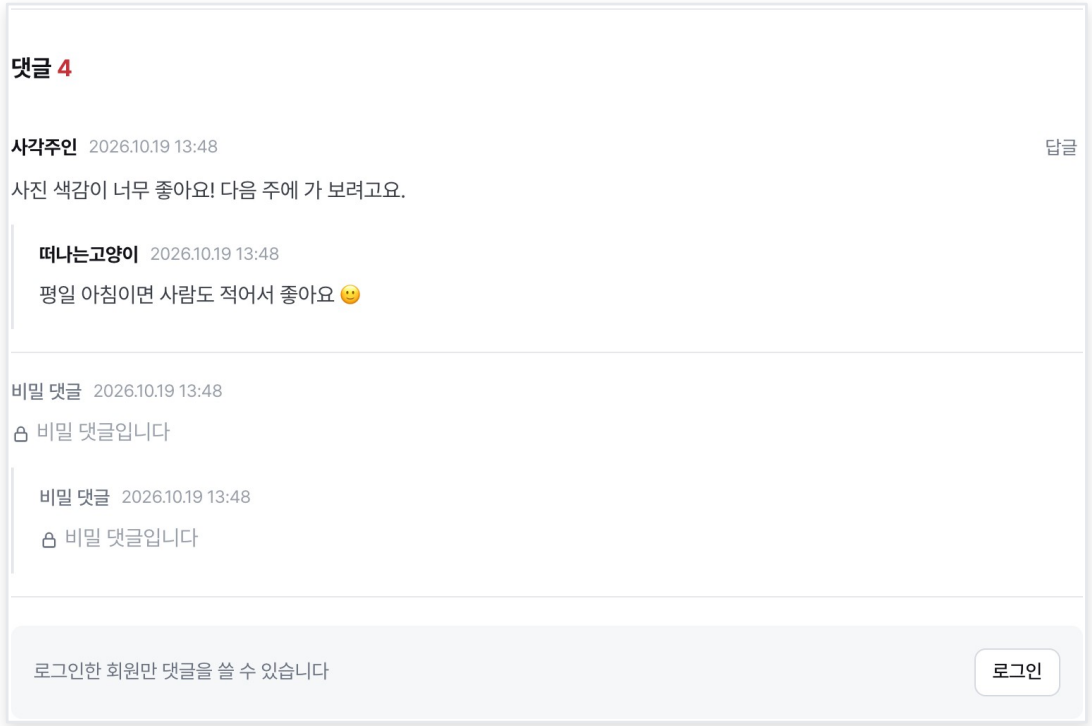
이번 주 인기 글 · 추천 글 · 인기 다이어리를 보여 준다. 누구에게나 같은 공개 화면이라 Redis에 30초 캐시하고, 글을 쓰면 바로 비운다.



② 글쓰기

마크다운으로 쓰고 미리보기로 확인한다. 다이어리 · 분야 · 공개 범위를 고르고, 60초마다 자동 임시저장한다. 본문 속 HTML은 그리지 않아 XSS를 막는다.

비밀 댓글 — 같은 글, 보는 사람에 따라 다르게



③ 손님 · 다른 회원이 볼 때

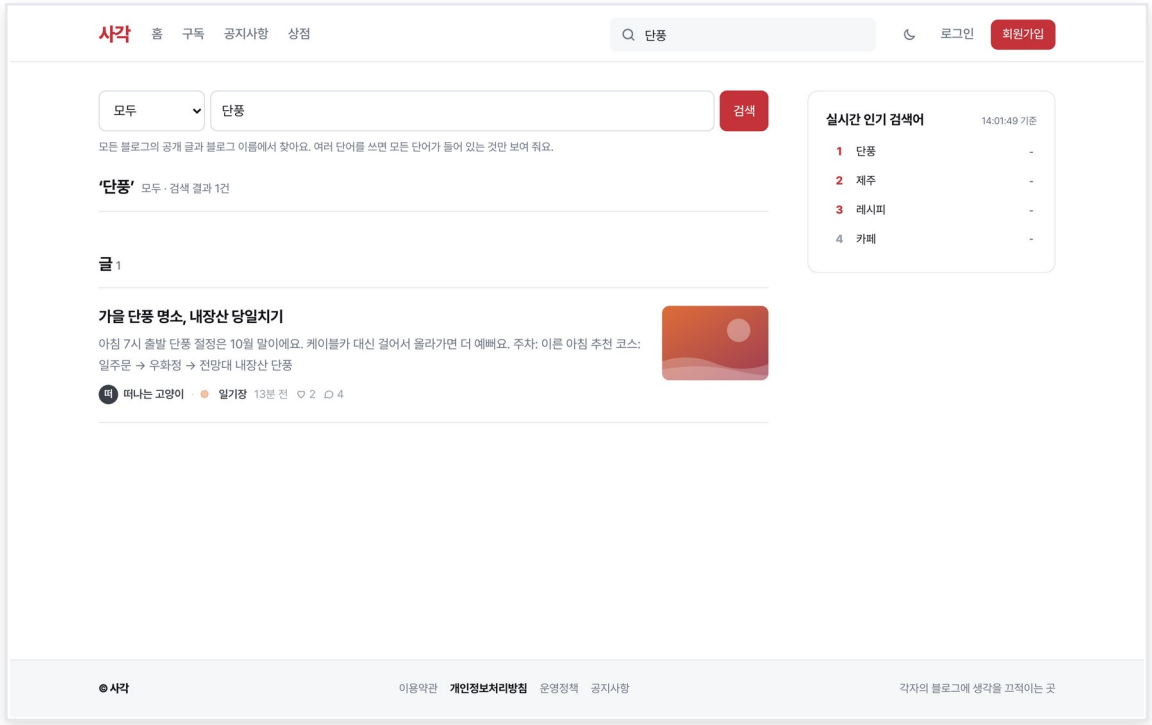
내용과 작성자가 가려지고 "비밀 댓글입니다"만 보인다. 서버가 응답에서 내용을 아예 빼서 개발자 도구로도 보이지 않는다.

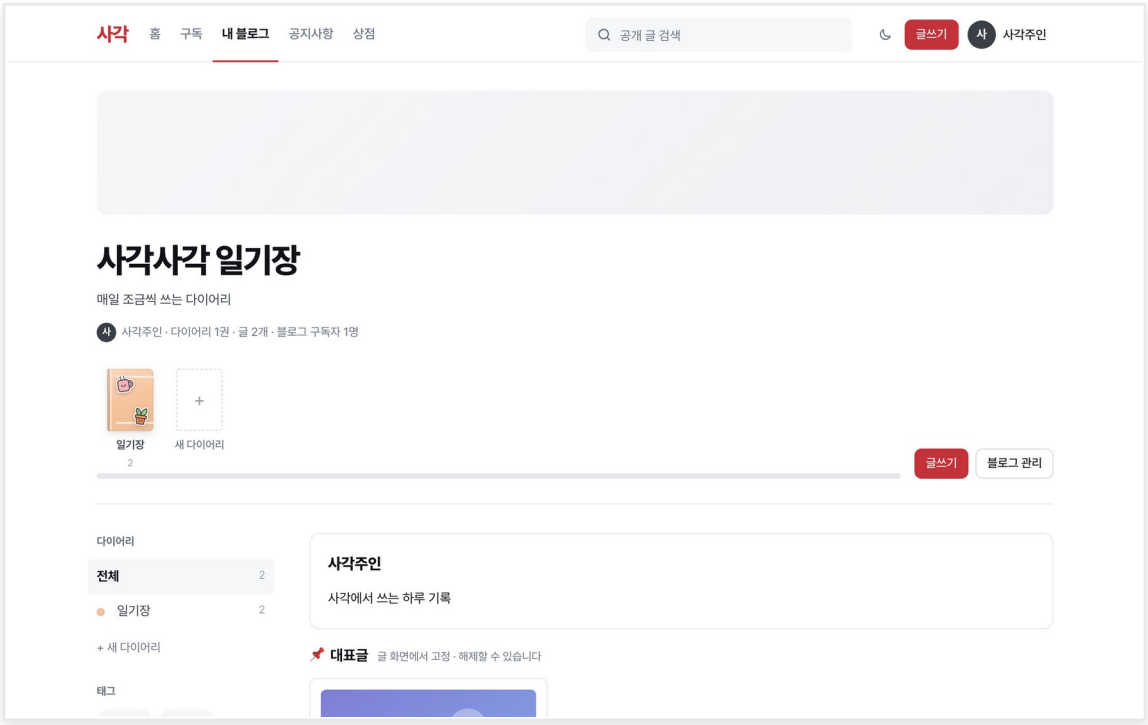


③ 글쓴이(블로그 주인)가 볼 때

쓴 사람 · 블로그 주인 · 원 댓글 작성자는 내용을 본다. 비밀 댓글에 단 답글은 체크하지 않아도 늘 비밀이다.

검색과 다이어리 꾸미기





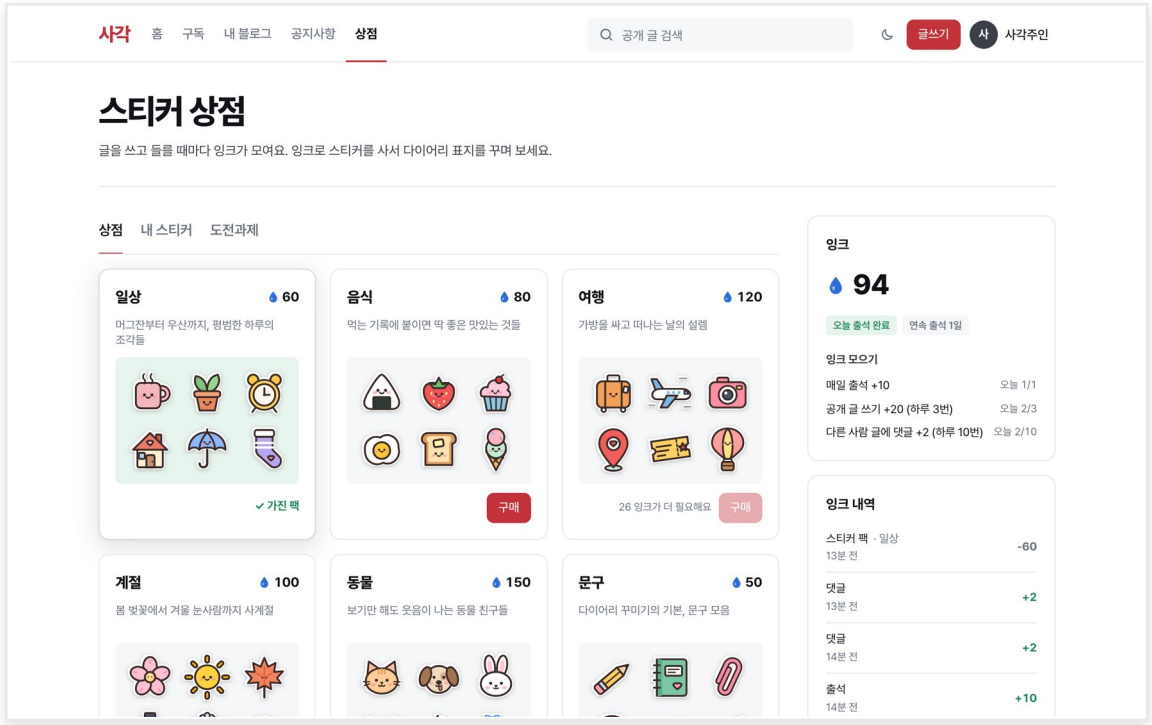
④ 통합 검색과 실시간 인기 검색어

여러 단어를 넣으면 모두 들어간 글만 찾는다(한국어 부분 일치, pg_trgm). 오른쪽 인기 검색어는 5초마다 다시 계산한다.

⑤ 블로그와 다이어리 표지

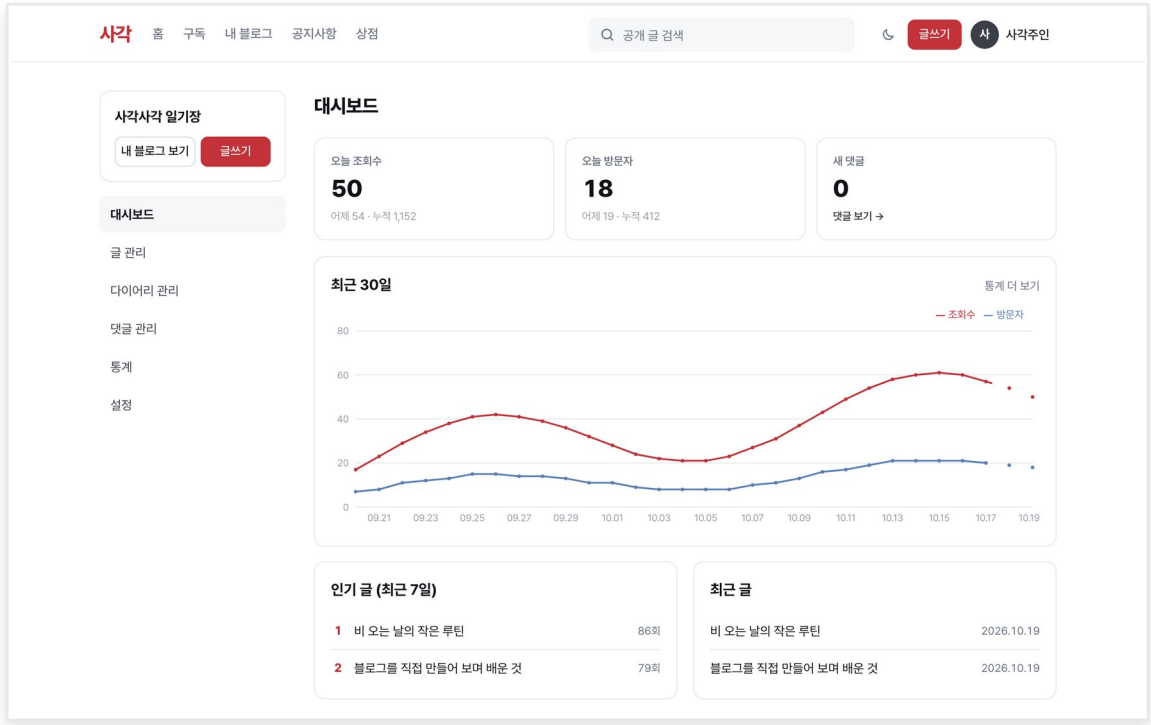
블로그 안에 다이어리를 여러 권 두고, 상점에서 산 스티커를 표지에 붙인다. 대표글은 3개까지 위에 고정한다.

스티커 상점과 블로그 관리



⑥ 스티커 상점

출석 +10 · 공개 글 +20(하루 3번) · 남의 글 댓글 +2(하루 10번)로 잉크를 모아 팩을 산다. 동시 요청에도 한도를 넘지 않게 지갑 줄을 잠근 뒤 센다.



⑦ 블로그 관리 대시보드

오늘 · 어제 · 누적 조회수와 방문자, 30일 그래프, 7일 인기 글을 한 화면에서 본다. 같은 사람의 조회는 30분에 한 번만 센다.

CI/CD와 Git 브랜치 전략 — 검사를 통과한 코드만 배포된다

feature/*

기능마다 브랜치

→

PR → develop

CI: 서버 테스트 · 화면 검사
· SonarQube 게이트

→

develop → main

릴리스 PR
+ E2E 7개

→

deploy.yml

CI → 도커 이미지
→ SSH → deploy.sh

→

배포 후 확인

실제 주소 · HSTS
확인, Release 생성

무엇을

GitHub Actions로 테스트 → 이미지 빌드 → 배포를 자동화하고, 간소화한 Git Flow(feature → develop → main)를 적용했다.

왜

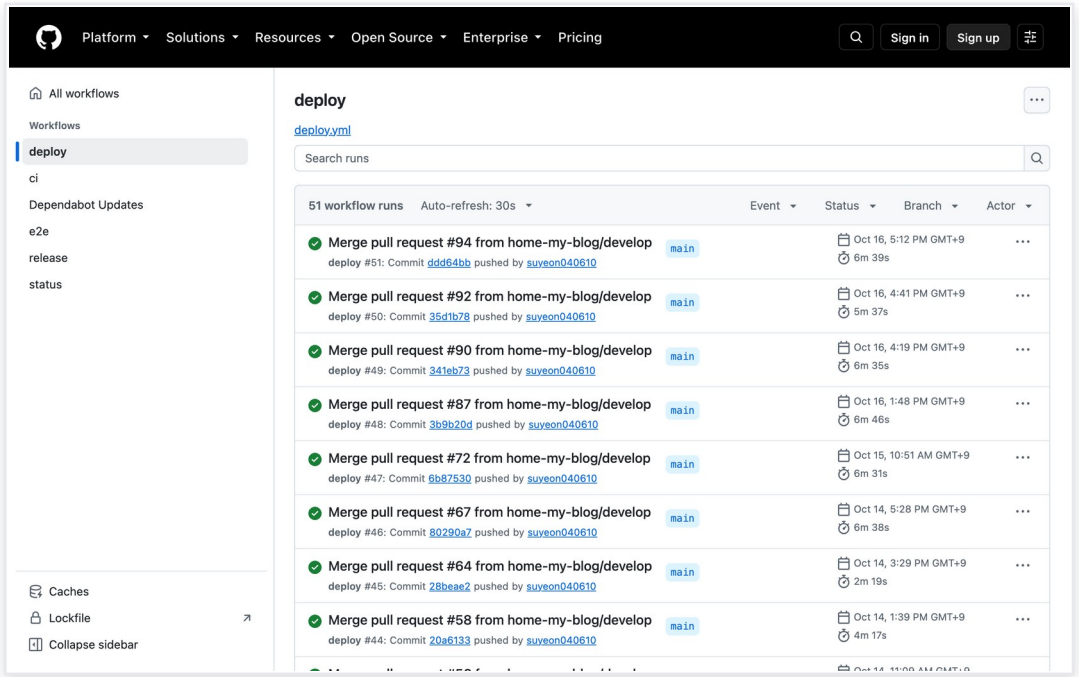
작은 수정마다 배포하면 너무 잦았고, 손으로 배포하면 테스트를 건너뛸 수 있었다. 혼자여도 "검사 통과 없이는 못 합친다"를 규칙이 아니라 시스템으로 만들고 싶었다.

어떻게

브랜치 규칙으로 develop · main은 backend · frontend · SonarCloud 세 검사가 통과해야 병합된다(관리자도 예외 없음). 비밀번호는 GitHub Secrets에만 둔다.

결과와 배운 점

자동 배포 51번 · 릴리스 21개. 기능 단위 PR이라 무엇이 언제 바뀌었는지 추적이 쉬웠고, CHANGELOG가 곧 릴리스 노트가 됐다.



무중단 배포와 리버스 프록시 — 끊기지 않고, 뒷문 없이

무엇을

Redis 세션 + nginx 블루-그린 배포. 새 서버를 옆에 띄워 정상 응답을 확인한 뒤에만 nginx를 전환하고, 옛 서버는 하던 요청을 끝내고 종료한다.

왜

배포 때마다 몇 초씩 끊기고, 세션이 서버 메모리에 있어 모든 회원이 로그아웃됐다. 새 버전이 안 뜨면 사이트가 내려간 채로 남았다.

어떻게

deploy.sh가 쉬는 색에 새 이미지를 띄우고 /api/topics 응답을 3초 간격 최대 3분 확인 → nginx -t → reload. 전문가 피드백으로 8390 포트를 127.0.0.1에만 열고 HSTS를 붙였다.

결과와 배운 점

0회	유지	차단
배포 중 끊김(0.2초 간격)	배포 전후 로그인	고장 난 버전 전환

대가: 두 버전이 잠깐 같은 DB를 써서 컬럼 삭제는 두 번에 나눠 배포한다.

블루-그린 전환 순서

1. 지금은 blue가 손님을 받는 중
2. green에 새 버전 컨테이너 실행
3. /api/topics가 200인지 확인 (최대 3분)
4. nginx 설정을 green으로 바꾸고 nginx -t
5. reload → 손님은 green으로
6. blue는 처리 중인 요청을 마치고 종료

롤링 대신 블루-그린: 무료 nginx엔 액티브 헬스 체크가 없어, 배포 스크립트가 직접 확인한 뒤에만 전환한다.

설계 — 로그인은 세션, 권한은 언제나 서버가

무엇을

JWT 대신 서버 세션 + HttpOnly 쿠키. 공개 범위와 비밀 댓글처럼 "누가 무엇을 보나"는 서버의 SQL 조건과 서비스 코드가 정한다.

왜

"비밀번호를 바꾸면 다른 기기도 바로 로그아웃" 요구사항은 JWT로는 어렵다. 화면에서만 숨기면 API를 직접 불러 내용을 볼 수 있다.

어떻게

CSRF는 XSRF-TOKEN 쿠키 + 헤더로 막고, 로그인 5회 실패 시 10분 잠금. 글 조회는 PUBLIC_ONLY / VISIBLE_TO 두 SQL 조각으로만 거르고, 못 보면 403 대신 404. 비밀 댓글은 서버가 body · author를 빼고 hidden만 보낸다.

결과와 배운 점

즉시
다른 기기 로그아웃

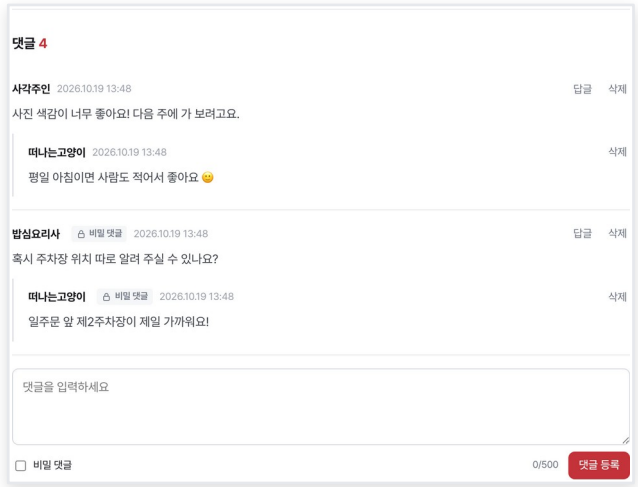
404
권한 없는 접근 응답

0
비밀 댓글 내용 노출

권한 규칙이 한곳에 있어 화면마다 어긋나지 않았고, 새 규칙은 서비스와 통합 테스트만 고치면 됐다.

CommentService.java — 비밀 댓글을 볼 수 있는 사람

```
static boolean canRead(CommentRow c, Long viewer) {
  if (!c.secret()) return true;       // 공개 댓글
  return viewer != null &&
    (viewer.equals(c.authorId())      // 쓴 사람
     || viewer == c.blogOwnerId()     // 블로그 주인
     || viewer.equals(c.parentAuthorId())); // 원 댓글
}
```



성능 개선 — 누구에게나 같은 화면은 30초 캐시

무엇을

메인 · 분야별 글 · 주제 화면 응답을 Redis에 JSON으로 30초 보관 (ResponseCache). 키에 세대 번호를 붙여 글이 바뀌면 번호만 올려 한꺼번에 비운다.

왜

메인을 한 번 열 때 쿼리가 7개, 인기 글은 글마다 좋아요 · 댓글 · 북마크를 세어 정렬한다. 글이 많아질수록 느려진다.

어떻게

무효화는 트랜잭션 커밋 뒤에. 캐시가 빈 순간 몰린 요청은 하나만 DB로 보낸다. 로그인한 사람마다 다른 화면(글 상세 · 마이페이지)은 캐시하지 않았다.

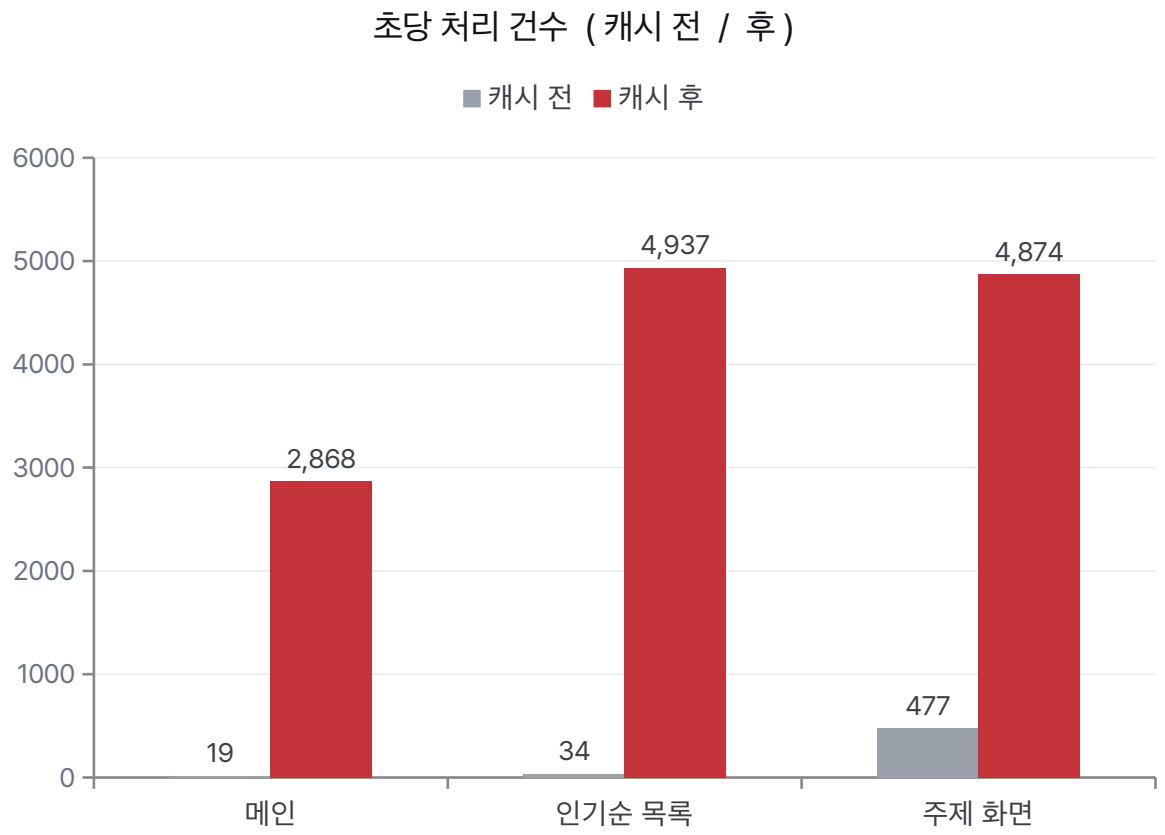
결과와 배운 점

154배
메인 처리량

3ms
응답 중앙값

30초
최대 반영 지연

Redis가 멈추면 캐시 없이 DB에서 읽어 서비스는 계속된다.



글 20,769개 · 동시 10명 · 응답 중앙값: 메인 485 → 3ms, 인기순 249 → 2ms

테스트 · 코드 품질 — 믿는 것은 AI가 아니라 테스트와 품질 게이트

무엇을

테스트 피라미드(단위 · 통합 · E2E) + SonarQube Cloud 품질 게이트(새 코드 버그 · 취약점 0, 커버리지 80%, 중복 3% 이하).

왜

AI가 쓴 코드가 많아 사람 리뷰만으로는 놓칠 수 있다. 서버 로직은 대부분 "권한 확인 + SQL"이라 실제 DB로 확인해야 의미가 있다.

어떻게

통합 테스트는 실제 PostgreSQL · Redis로, 입력 규칙은 단위 테스트로. 경계값은 양쪽(64 / 65자)을 본다. 화면 커버리지 34%는 측정에서 빼지 않고 사용 흐름 테스트 241개로 채웠다.

결과와 배운 점

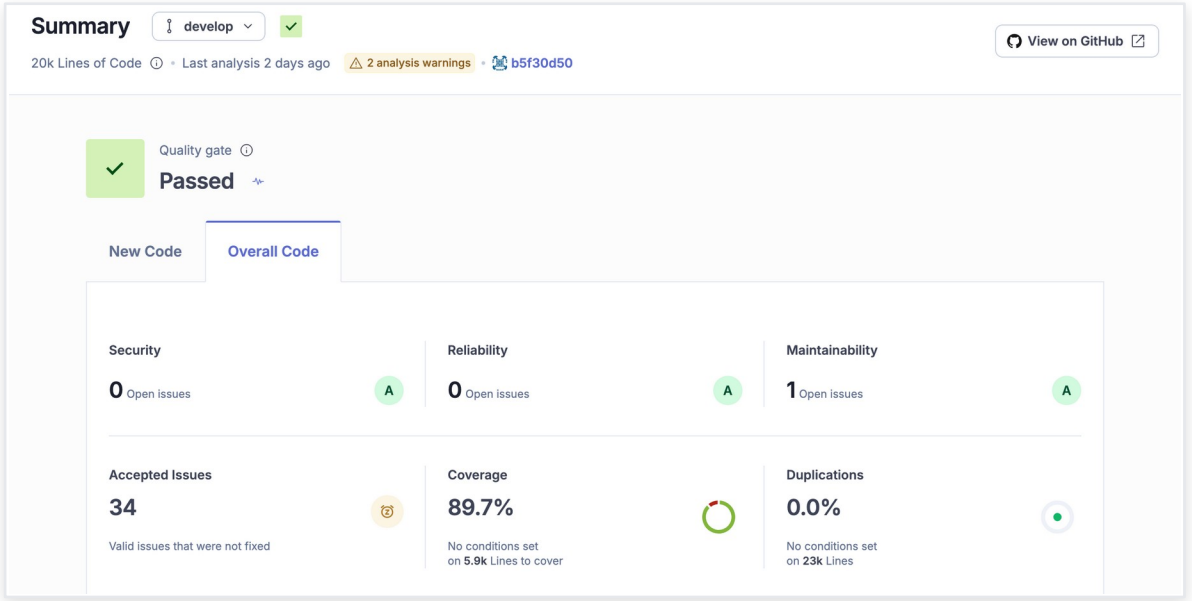
89.7%
커버리지

A
신뢰성 · 보안 · 유지보수성

229
정리한 이슈

게이트가 내 정규식 실수와 커버리지 61%를 병합 전에 막았다.





명세 · 문서화 — 코드보다 명세를 먼저

무엇을

Spec Kit으로 요구사항 → 명세(spec) → 계획(plan) → 작업(tasks)을 만들고, 프로젝트 원칙(constitution) 6가지를 먼저 정했다.

왜

요구사항 없이 AI에게 구현을 맡기면 그럴듯하지만 요구사항과 다른 코드가 나온다. 무엇이 정답인지 문서로 먼저 고정해야 결과를 검증할 수 있다.

어떻게

명세의 Given-When-Then이 곧 테스트 목록이 되고, 테스트 이름에 요구사항 번호(CF_13_4_...)를 붙였다. README · 기능 / 원리 문서 · API · ERD는 docs 저장소에서 따로 관리했다.

결과와 배운 점

6개	24개	21개
프로젝트 원칙	공통 요구사항(CF)	릴리스 노트

원칙이 AI 작업의 기준선이 됐고, 기능을 더할 때도 무엇이 바뀌는지 먼저 정리할 수 있었다.

I. 요구사항 문서가 기준이다 (Requirements Are the Source of Truth)

`docs/` 의 요구사항 문서(요구사항.md, 요구사항-공통.md, 상세/)가 무엇을 만들지의 기준이다(MUST). spec, plan, tasks, 코드, 테스트는 근거가 되는 원본 ID(`CF-05-3` , `BM-06-3` , `NF-07` 등)를 적어 추적할 수 있어야 한다(MUST). 원본과 구현이 다르면 코드를 원본에 맞추거나, 원본을 먼저 고친 뒤 구현한다(MUST). 화면 문구는 원본의 `안내 문구` 표를 그대로 쓴다(SHOULD).

II. 권한과 공개 범위는 서버가 지킨다 (Server-Enforced Access)

글·댓글·분류·블로그·관리 화면의 수정·삭제·접근 권한은 서버가 요청마다 검사한다(MUST, NF-02). 화면에서 버튼을 숨기는 것은 보조일 뿐이다. 비공개 글은 작성자 외 누구에게도 목록, 검색, 이전/다음 글, 홈, 주제 화면, 통계, 주소 접근 어디에서도 보이면 안 되고(MUST, CF-13-4), 없는 글과 같은 응답을 돌려준다(MUST, CF-09-4). 공개 글만 고르는 조건은 한 곳에 두고 모든 방문자용 조회가 그것을 쓴다(MUST).

III. 단순하게 시작한다 (Simplicity)

서버는 기능별 패키지로 나눈 **단일 Spring Boot 서버**다(MUST). MSA, JWT, 메시지 큐(RabbitMQ·Kafka), 검색 전용 서버(Elasticsearch)는 쓰지 않는다(기술스택-아키텍처.md 4장). 새 인프라 구성 요소는 spec의 요구사항을 그것 없이 지킬 수 없다는 근거를 plan.md에 적을 때만 들인다(MUST). 잃어도 되는 짧은 값만 Redis에 두고, 로그인은 Redis 없이도 동작해야 한다(MUST).

트러블슈팅 ① — 배포 · 운영에서 만난 문제

두 번째 요청부터 로그인이 풀림

문제

세션을 Redis로 옮긴 뒤, 로그인 다음 요청부터 401. 자동 테스트는 모두 통과한 상태였다.

원인

"7일 연장" 필터가 세션 ID를 base64 인코딩 없이 쿠키에 다시 담아, 서버가 세션을 찾지 못했다.

해결

쿠키 값 형식을 통일하고, 연장된 쿠키로 다시 요청하는 테스트를 추가했다.

결과

재발 방지 테스트 확보. 배포 전 가입 → 로그인 → 글쓰기 흐름을 확인하는 절차를 만들었다.

HTTPS를 건너뛰는 뒷문

문제

http://공인IP:8390 으로 들어오면 도메인과 HTTPS 없이 사이트가 열렸다.

원인

블루-그린용 nginx 컨테이너를 -p 8390:8390 으로 띄워 모든 주소에서 요청을 받았다.

해결

8390을 127.0.0.1에만 바인딩하고 HSTS를 붙였다. 주소가 바뀔 때만 컨테이너를 다시 띄운다.

결과

도메인(HTTPS)으로만 접속 가능. 외부망에서 IP:8390 차단을 확인했다.

맞는 비밀번호로 DB 접속 실패

문제

.env의 비밀번호로 서버를 켜면 password authentication failed.

원인

PostgreSQL 이미지는 비밀번호를 볼륨을 처음 만들 때만 적용한다. 예전 볼륨이 남아 있었다.

해결

데이터를 지우는 down -v 대신 ALTER USER로 DB 안의 비밀번호만 맞췄다.

결과

데이터 손실 없이 해결. 지우는 해결은 마지막 수단으로 둔다.

트러블슈팅 ② — 테스트 · 품질에서 만난 문제

커버리지가 화면에 안 보임

문제

PR에는 커버리지가 나오는데 SonarCloud 첫 화면(develop)은 비어 있었다.

원인

develop 병합 뒤에는 CI가 돌지 않았고, 자동 분석이 CI 분석과 충돌했다.

해결

자동 분석을 끄고(토큰 → 시크릿 → 끄기 순서), develop push에도 CI를 돌렸다.

결과

커버리지가 보이자 화면 34%가 드러났고, 테스트 241개로 94%까지 채웠다.

테스트가 가끔만 실패

문제

파일 하나만 돌리면 통과, 전체를 돌리면 가끔 실패했다.

원인

화면 이동 · 응답 반영이 끝나기 전에 다음 확인을 하는 경쟁 상태였다.

해결

대기 시간을 늘리지 않고 "무엇이 끝났는지"를 기다리는 조건으로 바꿨다.

결과

전체 연속 5회 실행 모두 통과. 원칙을 TESTING.md에 남겼다.

긴 입력에 느려지는 정규식

문제

SonarCloud가 이메일 검사식을 ReDoS 위험으로 지적했다.

원인

@ 뒤 두 부분이 같은 글자(점)를 나눠 먹어, 틀린 긴 입력에서 되돌아가기가 폭증한다.

해결

도메인을 점으로 나뉜 조각으로 바꾸고 소유 수량자(++)를 써서 되돌아가지 않게 했다.

결과

10만 자 입력도 즉시 판정. 덤으로 a@b..c 같은 틀린 주소도 막는다.

사용한 AI 도구와 역할 분담 — AI는 페어 프로그래머, 결정은 내가

도구	쓴 곳
Claude Code	스펙 · 작업 목록 초안, 서버 · 화면 코드와 테스트, PR 작성, 배포 확인, 문서 · README 초안
Claude (대화)	수업 내용 정리, 기술 선택지 비교 질문, 노션 공부 노트 정리
Spec Kit	요구사항 → spec → plan → tasks 명세 흐름
MCP 연결	Notion(공부 노트), Crowfoot(ERD 문서), GitHub CLI(PR · 릴리스)

왜 Claude Code인가: 저장소 전체를 읽고, 테스트를 직접 돌려 결과로 확인하며, PR 단위로 작업을 나눠 주기 때문에 "검증 가능한 단위"로 일을 맡길 수 있었다.

누가 무엇을 했나

나

- 요구사항 · 기능 범위 · 우선순위 결정
- 기술 선택 (대안 비교 후 최종 결정)
- 모든 PR 검토 · 병합 · 배포 승인
- 실제 사이트에서 기능 검증 · 문제 발견

Claude Code

- 스펙 · 작업 목록 초안
- 코드와 테스트 작성
- 문서 · 릴리스 노트 초안
- 검사 실패 원인 분석

단계별로 어떻게 썼나 — 그리고 내가 고친 부분

기획 · 명세

공통 요구사항 문서를 넣고 Spec Kit으로 spec · plan · tasks 초안을 받았다.

내가 고친 것 처음 명세는 일반 블로그 기준이라, 내가 정리한 요구사항 문서를 기준으로 다시 쓰게 했다. 프로젝트 원칙 6가지를 정해 이후 작업의 기준으로 삼았다.

코드 작성

기능 단위로 "정상 · 경계 · 예외 테스트를 먼저 정하고 구현"하도록 맡겼다. 화면이 바뀌는 작업은 시안을 먼저 보고 승인했다.

내가 고친 것 무엇을 만들지(비밀 댓글 추가)는 내가 정하고, 사용자 편의성을 고려해 아이콘을 사이트 톤에 맞게 통일했다.

리뷰 · 테스트 · 디버깅

CI 실패 · SonarQube 지적의 원인 분석과 수정안을 받았다.

내가 고친 것 지적을 일괄로 고치지 않고 규칙마다 우리 코드에 맞는지 판단하게 했고, 오탐은 이유를 기록해 처리했다.

작업 환경 설정

CLAUDE.md에 꼭 지킬 규칙을, TESTING.md에 테스트 전략을 적어 AI가 매번 읽게 했다.

내가 고친 것 테스트를 고쳐서 통과시키지 않기, develop · main 직접 push 금지를 규칙으로 고정했다.

사각 개발 규칙 (Claude Code용)

- 프로젝트 원칙: `.specify/memory/constitution.md`
- 테스트 전략과 AI 규칙: `TESTING.md` — 기능을 만들거나 고치기 전에 읽고 따른다.

꼭 지킬 것

- 기능을 만들 때 정상 · 경계 · 예외 테스트를 먼저 정하고, 그 테스트를 통과하도록 구현한다.
- 테스트가 실패하면 구현을 고친다. 테스트를 지우거나 건너뛰거나 기대값을 바꿔 통과시키지 않는다.
- 새 코드는 SonarQube 품질 게이트(새 버그 · 취약점 0, 커버리지 80% 이상, 중복 3% 이하)를 넘어야 병합한다.
- 모든 변경은 `feature/*` → `develop` PR로. `develop` · `main`에 직접 push하지 않는다.
- 비밀 값(비밀번호, 토큰, 키)은 코드에 넣지 않는다. GitHub Secrets와 서버 환경 변수로만.

확인 명령

```
cd backend && ./gradlew test                                    # 서버 (Postgr
cd frontend && npm run lint && npm run typecheck && npm test  # 화면
cd frontend && E2E_BASE_URL=http://localhost:8080 npm run e2e # 종단 (서버를 MY
```

CLAUDE.md — Claude Code가 작업을 시작할 때마다 읽는 규칙 파일

활용 사례와 검증 — AI 결과를 그대로 쓰지 않았다

영역	AI에 맡긴 방향	검증 · 결과
테스트 커버리지 보강	측정 대상에서 파일을 빼는 대신, 실제 사용 흐름(입력 · 클릭 · 서버 요청)을 확인하는 테스트를 쓰도록 지시	화면 테스트 241개 추가, 커버리지 34% → 94% (전체 89.7%)
정적 분석 지적 처리	일괄 수정 대신 규칙마다 우리 코드에 맞는지 판단하도록 지시	이슈 229개 정리, 맞지 않는 지적은 사유를 기록해 오탐 처리
보안 취약점 수정	전문가 피드백을 바탕으로 앱 포트 노출 여부를 점검하고 수정하도록 요청	127.0.0.1 바인딩 + HSTS, 외부망에서 차단 확인 (v1.10.1)
UI 일관성 개선	사용자 편의성 관점에서 비밀 댓글 표시를 사이트 아이콘과 통일하도록 요청	선 모양 SVG 자물쇠로 교체, 테스트 · 품질 게이트 통과 후 배포 (v1.11.1)

AI가 놓친 것을 잡은 방법

배포 후 흐름 점검 자동 테스트를 통과한 세션 버그를 실제 흐름에서 발견 → 재요청 테스트 추가

품질 게이트 AI가 수정한 정규식의 남은 위험과 부족한 테스트를 병합 전에 차단

테스트 우선 규칙 기존 테스트가 깨지면 기대값이 아니라 구현을 수정

실제 대화 — 내가 정하고, AI가 구현하고, 내가 확인한다

Claude Code · 비밀 댓글 작업 (2026.10.16) — 실제 작업 흐름을 요약해 정리

비밀 댓글 기능을 추가해 줘.
· 조건: 기존 댓글 API와 호환, 블루-그린 배포 중에도 안전한 DB 변경
· 완료 기준: 정상 · 경계 · 예외 테스트와 품질 게이트 통과 → 머지 · 배포 → ERD · README · 문서 반영

Claude Code

공개 범위 규칙을 먼저 정했습니다.
· 열람: 작성자 · 블로그 주인 · (답글이면) 원 댓글 작성자
· 그 외에는 응답에서 body · author를 빼고 hidden=true만 전달
· 비밀 댓글의 답글은 강제로 비밀
DB는 comments.secret(V25) 컬럼 추가만이라 블루-그린 전환 중에도 호환됩니다.

Claude Code

테스트 10개(서버 5 · 화면 5)를 먼저 작성했습니다. 기존 테스트 2개가 실패해 기대값이 아니라 구현을 고쳤습니다(secret은 true일 때만 전송).
CI · SonarCloud 통과 → PR #88 · #90 병합, v1.11.0 배포와 배포 후 점검까지 완료했습니다.

배포 화면을 검토해 보니 비밀 댓글 표시만 이모지라 사이트 아이콘(선 아이콘)과 톤이 달라. 같은 스타일의 SVG로 통일하고 야간모드에서도 글자 색을 따르게 해 줘. 기존 테스트는 유지하고 v1.11.1로 배포.

Claude Code

LockIcon(SVG, currentColor) 컴포넌트로 교체했습니다. 화면 테스트 297개 · 품질 게이트 통과 후 v1.11.1로 배포했습니다.

이 대화에서 보이는 것

요구 범위는 내가 정의했다

기능과 완료 기준(DB · ERD · 테스트 · 배포 · 문서)을 먼저 정하고 맡겼다.

AI는 설계 → 테스트 → 배포를 실행했다

권한 규칙과 하위 호환 DB 변경을 제안하고, 테스트 우선 원칙대로 구현해 게이트를 통과한 것만 배포했다.

결과는 사용자 관점에서 검토했다

배포 화면을 검토해 UI 일관성 문제를 찾아 개선을 지시했고, 같은 절차로 v1.11.1을 배포했다.

이 보고서도 Claude와 함께 — 가이드 4단계대로

1 프로젝트 분석

양식과 안내문을 먼저 분석하게 하고, 코드 · README · CHANGELOG · 커밋 기록에서 기술 스택 · 버전 · 구조를 뽑게 했다.

2 기술적 고민 끌어내기

프로젝트를 하며 내린 판단(품질 지적 처리 방식, 포트 노출 보안 수정 등)을 무엇 / 왜 / 어떻게 / 결과로 정리했다.

3 트러블슈팅 정리

README에 남긴 7건 중 5건을 문제 → 원인 → 해결 → 결과 표로 옮겼다.

4 슬라이드 만들기

양식의 목차 5개를 지키고 사이트 색 · 글꼴(Pretendard)로 디자인. 기능 화면은 로컬 데모 데이터로 직접 띄워 캡처했다.

발표할 때 내 말로 설명할 수 있도록, 슬라이드마다 실제로 한 일과 맞는지 확인했다.

그대로 붙이지 않았다

- 초안을 PPT · PDF로 직접 열어 확인한 뒤 브랜치에 올렸다
- 마음에 들지 않는 글꼴은 바꾸게 했다
- 완성도 점수와 느낀 점은 내 생각과 맞는지 다시 확인했다
- 수치(154배 · 0회 · 89.7%)는 README · 측정 기록 · SonarCloud 화면과 맞는지 확인했다

효과와 한계 — 다음에는 이렇게 쓰겠다

21

릴리스 (9일)

83

병합한 PR

431

자동 테스트 (서버 127 · 화면 297 · E2E 7)

효과

- 반복 작업(테스트 · 문서 · 릴리스 노트)이 빨라져, 기획 · 검증 · 판단에 시간을 썼다
- 기능 하나를 명세 → 테스트 → PR → 배포 → 문서까지 하루 안에 끝냈다(비밀 댓글)
- 품질 게이트와 테스트가 있어 큰 정리(이슈 229개)도 안심하고 맡겼다

한계와 주의할 점

- 그럴듯하게 틀린다: 테스트가 다루지 않은 경우(쿠키 재요청)는 그대로 통과했다
- AI 세션 두 개가 같은 작업을 해서 PR이 겹친 적이 있다
- 비밀번호 · 토큰 · 저장소 설정처럼 사람이 직접 해야 하는 일은 남는다

다음 프로젝트에서는

- 테스트 전략 · 품질 게이트 · 보안 체크리스트를 첫날부터 켠다
- 작업을 나눌 때 세션마다 맡을 범위를 먼저 정한다
- AI가 만든 결과는 실제 화면 · 실제 요청으로 한 번 더 확인한다

한 줄 정리 — AI는 속도를 맡고, 신뢰는 테스트 · 품질 게이트 · 직접 검증이 맡았다.

완성도 평가와 잘한 점 · 아쉬운 점

완성도

9 / 10

기획한 기능을 모두 실제 사이트에 배포했고, 무중단 배포 · 품질 게이트 · 테스트까지 갖췄다. 푸시 알림 · 신고 처리가 아직 없어 −1점.

잘한 점

- 기능을 "만들었다"에서 멈추지 않고 실제 도메인에서 운영하며 문제를 찾아 고쳤다
- "왜 이 기술인가"를 대안과 비교해 고르고 문서(README 의사결정 11개)로 남겼다
- AI 결과를 그대로 믿지 않고 테스트 · 품질 게이트 · 직접 검증으로 확인했다
- 숫자로 말할 수 있는 결과: 154배, 끊김 0회, 커버리지 89.7%

아쉬운 점

- 테스트 전략을 처음부터 세우지 않아, 화면 커버리지 34%를 뒤늦게 몰아서 채웠다
- 앱 포트 노출을 배포 뒤 수업에서 배우고서야 알았다
- 서버 한 대 · 앱 하나라 서버가 멈추면 사이트도 멈춘다(단일 장애점)
- 신고는 저장만 되고 처리 화면이 없다

05 자체 평가 의견

앞으로 개선할 점과 느낀 점

개선할 점

1. **가용성**
   앱을 두 개 띄워 로드 밸런싱 (세션은 이미 Redis라 코드 변경 없음)

2. **저장소**
   사진을 S3 호환 저장소로 (인터페이스는 이미 분리), 이미지는 레지스트리로 배포

3. **운영 기능**
   신고 처리 화면, 새 글 · 댓글 알림

4. **보안**
   IP 단위 요청 제한, Content-Security-Policy 헤더

느낀 점

AI와 함께 개발하면 코드를 쓰는 속도는 크게 빨라지지만, "무엇이 맞는지"를 정하고 확인하는 일은 여전히 내 몫이었다.

그래서 요구사항 · 테스트 · 품질 게이트 · 실제 사이트 검증처럼 기준을 먼저 세우는 일이 오히려 더 중요해졌다. 9일 동안 21번 배포하며, 좋은 개발자는 많이 쓰는 사람이 아니라 기준을 세우고 검증하는 사람이라는 걸 배웠다.

앞으로도 "왜 이렇게 했는지"를 설명할 수 있는 개발자, 만든 것을 끝까지 운영하고 고치는 개발자가 되고 싶다.

감사합니다

사이트 https://sagak.java21.net

GitHub github.com/home-my-blog/myblog

문서 github.com/home-my-blog/docs

박수연 · AIGJ_01_010